Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

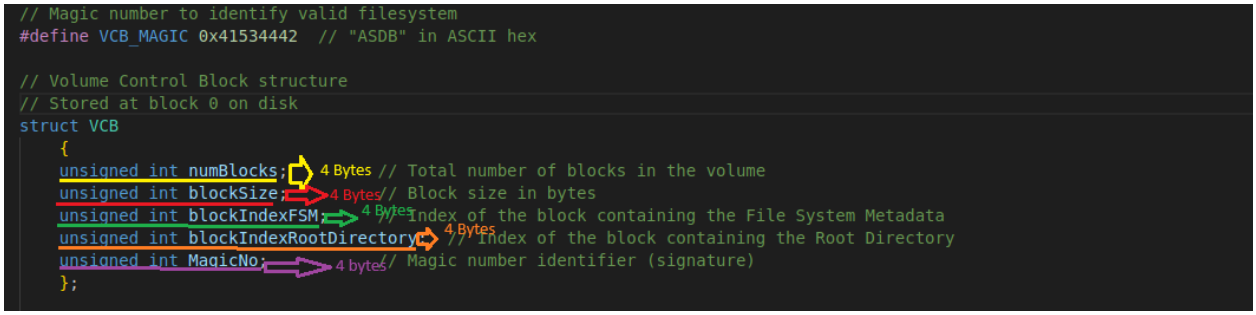
Don’t Worry We Don’t Byte

Member	StudentID	Github name
Ian Kligman	924122742	ikligman
Joe Bowen	922966782	josephb333
Cheng-Chieh Yao	923978442	Jamesyao0818
Enrique Cabral-Mixco	924056104	rommix408

https://github.com/CSC415-2025-Fall/csc415-filesystem-josephb333

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Volume Control Block Description



Yellow = Num Blocks; Total # of blocks in volume.
Red = Block Size; In bytes
Green = Block index FSM; Index in LBA
Orange = BlockIndexRootDirectory; Index in LBA
Purple = MagicNo; Signature which we #defined to be hex-decimal corresponding to ACII “ASDB”, 1095976002 in decimal.

The VCB struct itself is formatted such that it holds all the necessary information we need (at least for milestone one).

The only additional necessary functions including an initialize function, a way to write/read the VCB to/from disk, and a fetcher function for use in initFSM().

```
// Initialize a new VCB and write it to disk (for formatting new
// Returns 0 on success, -1 on error
int initVCB(uint64_t numberOfBlocks, uint64_t blockSize);

// Read VCB from disk (block 1) into memory
// blockSize is needed to allocate the correct buffer size
// Returns 0 on success, -1 on error
int readVCB(uint64_t blockSize);

// Write VCB from memory to disk (block 1)
// Returns 0 on success, -1 on error
int writeVCB(void);

// Get pointer to in-memory VCB (for other modules to access)
// Returns NULL if VCB not loaded
struct VCB* getVCB(void);
```

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Free Space Map Description

We decided to use a bitmap to keep track of free space. The first functions to implement were getBit, setBit, and clearBit so that they can be utilized by the initialize and allocate functions. These functions are static because they are meant to only be called by initFSM and allocate. They use bitwise logic operations with bitmasks to single out 1 bit.

The fsm char * will be maintained in memory as a global static variable. That way, each allocate call does not involve reading/writing to the disk. To keep the in memory version up to date with disk, callers can use fsmWrite after making changes (such as repeated calls to allocate) which writes the entire 5 blocks of fsm to disk.

The allocate function does not need to return a starting block number of a grouping of contiguous available blocks because our files are going to use discontiguous linked lists. This means allocate simply finds the next free block, marks it as used, and returns that block number for the caller to use.

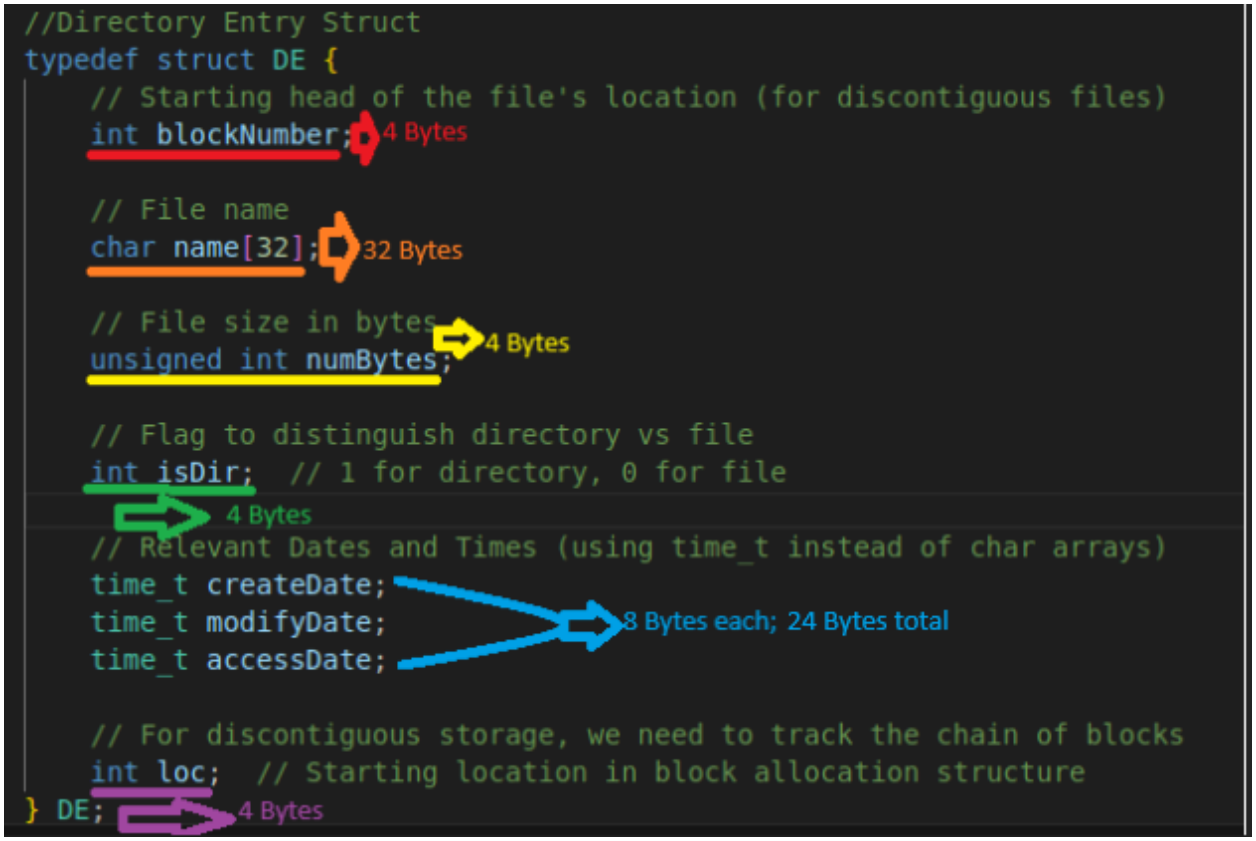
The initFSM function is what mallocs the global static fsm, sets all blocks to 0 (free), then marks block 1 as used (VCB) and blocks 2-6 as used (FSM), marks block 7 as used (Root directory), and writes the map to disk.

The loadFSM function allows the initFS function to simply load a pre-existing fsm from disk into memory in the event that a matching signature is detected on startup. This is also important during an initial formatting of a volume, as the init functions all get called first (including initFSM, which writes to disk). However after that, the system will attempt to load that newly initialized fsm into memory.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Description

Here is the visual reference of the Directory Entry structure.



RED = Block Number; corresponds to the head of the file’s location.

ORANGE = name; Is the name of the file.

YELLOW = numBytes; Is the file’s size in bytes.

GREEN = isDir; Is a sentinel value used as a flag to say whether a file is a directory or not.

BLUE = createDate, modifyDate, accessDate; Are timestamps to keep track of creation, last modification, and last access. (Set as the same for milestone one).

PURPLE = Location; Is a separate field then block number. So that we can track the next segment of a file that is larger than 1 block via discontiguous allocation. (To be implemented)

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Description

```
DE* createDir(int minEntries, DE * parent, int blockSize) {
    //4a) How much memory neeeded
    //4b) Number of entries wanted is minEntries parameter.
    //4c) Multiply size of your directory entry by number of entry to get number of bytes needed.
    // Make sure we never have less then 2 directories.
    if (minEntries < 2) { minEntries = 2;}
    int bytesNeeded = minEntries * (int)sizeof(DE);
    //4d) How many blocks needed.
    int blocksNeeded = (bytesNeeded + (blockSize - 1)) / blockSize;
    int bytesToMalloc = blocksNeeded * blockSize;
    // How many entries could fit in that space
    int actualEntriesCount = bytesToMalloc / (int)sizeof(DE);
    if (blocksNeeded != 1) { However,
        //Multiple Block Dirs not implemented yet.
        return NULL;
    }

    DE * dir = (DE *)malloc(bytesToMalloc);
    if (!dir) { return NULL;}
    //Make clean slate.
    memset(dir, 0, bytesToMalloc);

    //4e) Initiliaze entries
    // Initialize all entries as empty
    for (int i = 2; i < actualEntriesCount; i++) {
        dir[i].name[0] = '\0';
        dir[i].isDir = 0;
        dir[i].numBytes = 0;
        dir[i].blockNumber = -1;
        dir[i].loc = -1;
    }

    time_t now = time(NULL);
    //4f) Allocate blocks (one block for root directory)
    int rootPhys = allocate(blockSize);
    if (rootPhys < 0) { free (dir); return NULL;}

    // Setup "." entry (current directory)
    strcpy(dir[0].name, ".");
    dir[0].numBytes = actualEntriesCount * (int)sizeof(DE);
    dir[0].isDir = 1;
    dir[0].createDate = now;
    dir[0].modifyDate = now;
    dir[0].accessDate = now;
    dir[0].loc = rootPhys;
    dir[0].blockNumber = rootPhys;

    // Setup ".." entry (parent directory)
    if (!parent) {
```

To create a directory entry. We have a function createDir() that returns a pointer to our DE struct. It takes in minEntries as parameter, the parent directory, and the block size.

It first does a quick safety check to make sure there are at least two entries to account for parent directory and child.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Description

```
DE* createDir(int minEntries, DE * parent, int blockSize) {
    //4a) How much memory neeeded
    //4b) Number of entries wanted is minEntries parameter.
    //4c) Multiply size of your directory entry by number of entry to get number of bytes needed.
    // Make sure we never have less then 2 directories.
    if (minEntries < 2) { minEntries = 2;}
    int bytesNeeded = minEntries * (int)sizeof(DE);
    //4d) How many blocks needed.
    int blocksNeeded = (bytesNeeded + (blockSize - 1)) / blockSize;
    int bytesToMalloc = blocksNeeded * blockSize;
    // How many entries could fit in that space
    int actualEntriesCount = bytesToMalloc / (int)sizeof(DE);
    if (blocksNeeded != 1) { However,
        //Multiple Block Dirs not implemented yet.
        return NULL;
    }

    DE * dir = (DE *)malloc(bytesToMalloc);
    if (!dir) { return NULL;}
    //Make clean slate.
    memset(dir, 0, bytesToMalloc);

    //4e) Initiliaze entries
    // Initialize all entries as empty
    for (int i = 2; i < actualEntriesCount; i++) {
        dir[i].name[0] = '\0';
        dir[i].isDir = 0;
        dir[i].numBytes = 0;
        dir[i].blockNumber = -1;
        dir[i].loc = -1;
    }

    time_t now = time(NULL);
    //4f) Allocate blocks (one block for root directory)
    int rootPhys = allocate(blockSize);
    if (rootPhys < 0) { free (dir); return NULL;}

    // Setup "." entry (current directory)
    strcpy(dir[0].name, ".");
    dir[0].numBytes = actualEntriesCount * (int)sizeof(DE);
    dir[0].isDir = 1;
    dir[0].createDate = now;
    dir[0].modifyDate = now;
    dir[0].accessDate = now;
    dir[0].loc = rootPhys;
    dir[0].blockNumber = rootPhys;

    // Setup ".." entry (parent directory)
    if (!parent) {
```

Next we do a bit of arithmetic to determine the correct amount of memory to malloc and the amount of entries to fill the block we insert it in.

Next it memsets the newly malloc memory to create a clean slate to work with.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Description

```
DE* createDir(int minEntries, DE * parent, int blockSize) {
    //4a) How much memory neeeded
    //4b) Number of entries wanted is minEntries parameter.
    //4c) Multiply size of your directory entry by number of entry to get number of bytes needed.
    // Make sure we never have less then 2 directories.
    if (minEntries < 2) { minEntries = 2;}
    int bytesNeeded = minEntries * (int)sizeof(DE);
    //4d) How many blocks needed.
    int blocksNeeded = (bytesNeeded + (blockSize - 1)) / blockSize;
    int bytesToMalloc = blocksNeeded * blockSize;
    // How many entries could fit in that space
    int actualEntriesCount = bytesToMalloc / (int)sizeof(DE);
    if (blocksNeeded != 1) { However,
        //Multiple Block Dirs not implemented yet.
        return NULL;
    }

    DE * dir = (DE *)malloc(bytesToMalloc);
    if (!dir) { return NULL;}
    //Make clean slate.
    memset(dir, 0, bytesToMalloc);

    //4e) Initiliaze entries
    // Initialize all entries as empty
    for (int i = 2; i < actualEntriesCount; i++) {
        dir[i].name[0] = '\0';
        dir[i].isDir = 0;
        dir[i].numBytes = 0;
        dir[i].blockNumber = -1;
        dir[i].loc = -1;
    }

    time_t now = time(NULL);
    //4f) Allocate blocks (one block for root directory)
    int rootPhys = allocate(blockSize);
    if (rootPhys < 0) { free (dir); return NULL;}

    // Setup "." entry (current directory)
    strcpy(dir[0].name, ".");
    dir[0].numBytes = actualEntriesCount * (int)sizeof(DE);
    dir[0].isDir = 1;
    dir[0].createDate = now;
    dir[0].modifyDate = now;
    dir[0].accessDate = now;
    dir[0].loc = rootPhys;
    dir[0].blockNumber = rootPhys;

    // Setup ".." entry (parent directory)
    if (!parent) {
```

We then initialize the entries with empty values. Create a time_t variable to get current time. Get the block number with which we can get the root directory. Now we assign fields of the child “.” directory and the parent “..”.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Description

```
    // Setup ".." entry (parent directory)
    if (!parent) {
        // Root directory - parent points to itself
        parent = dir;
    }

    //4h)
    int parentPhys = parent->blockNumber;

    strcpy(dir[1].name, "..");
    dir[1].numBytes = parent->numBytes;
    dir[1].isDir = 1;
    dir[1].createDate = dir[0].createDate;
    dir[1].modifyDate = dir[0].modifyDate;
    dir[1].accessDate = dir[0].accessDate;
    dir[1].blockNumber = parentPhys;
    dir[1].loc = parentPhys;
    //4i)
    writeDirToDisk(dir, blockSize);
    //4j)
    return dir;
}
```

Then we write the directory entries to disk using writeDirToDisk().

```
void writeDirToDisk(const DE * dir, int blockSize) {
    //This version of writing directory is only writing one block.
    //For next milestone we will implement linked functionality to make it discontiguous.
    //Remember to refactor this function LATER!!!!
    int blocksToWrite = (dir[0].numBytes + blockSize - 1) / blockSize; // Should just be one.
    int blockIndex = dir[0].blockNumber;
    LBAwrite((char*)dir, 1, blockIndex);
    /*for (int i = 0; i < blocksToWrite; i++){
        //int blockIndex = getNthBlockOfChain(dir->loc,i);

        LBAwrite((char*)dir + i*blockSize, 1, blockIndex);
        //char * [numBytes...loc....blocknumber...]
    }*/
    fsmWrite();
}
```

Our writeDirToDisk() function uses a pointer to the DE structure and block size to LBAwrite() the one block necessary at the appropriate free block we got earlier with allocate().
**Note the comments; the functionality currently does not support our planned discontiguous allocation but will in the future once we implement a linked list and getNthBlockOfChain() functionality.*

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Volume Control Block Hex Dump

In order to determine if our hex dump is correct we will first analyse the layout of our struct’s fields with corresponding byte lengths.

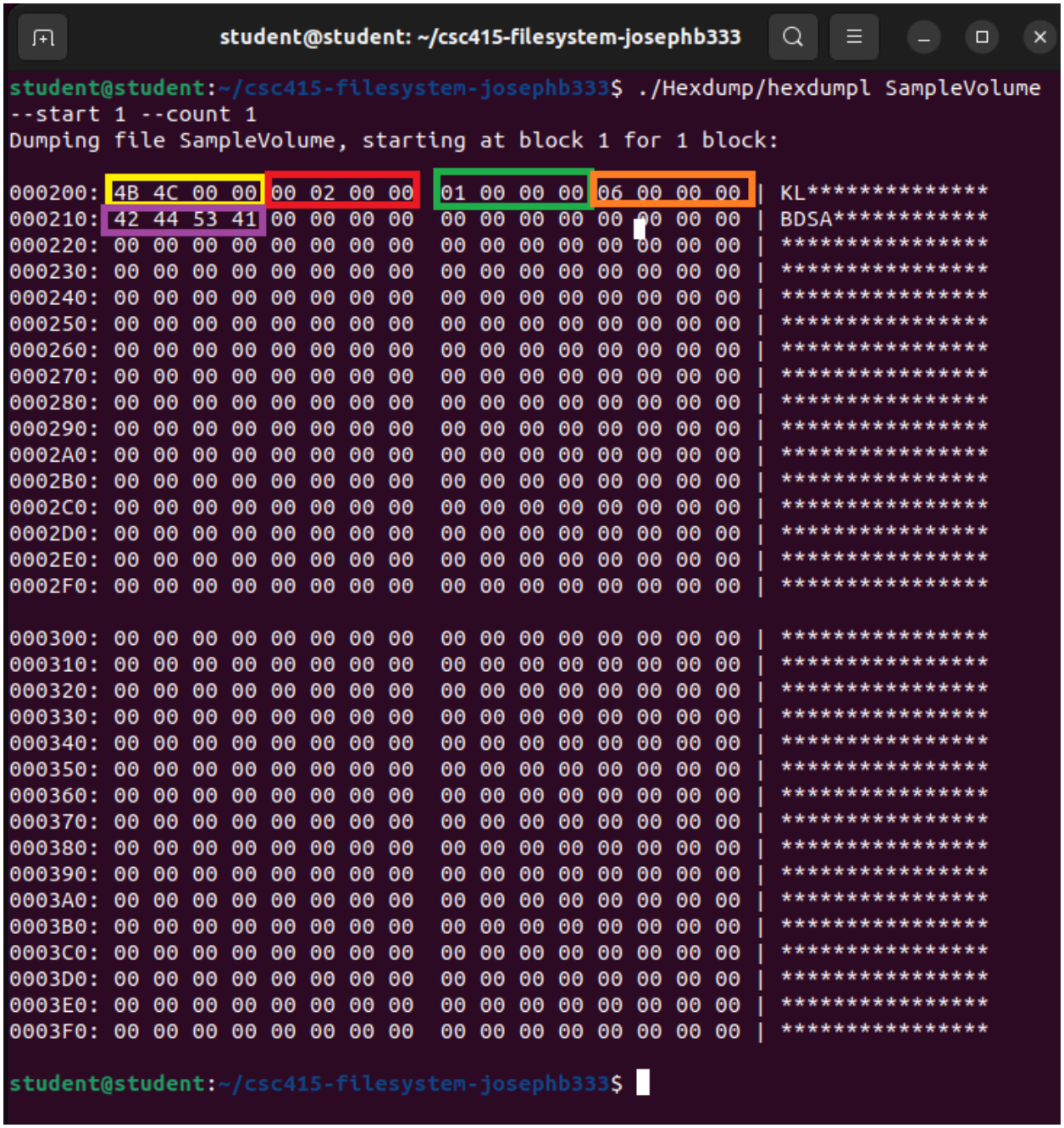
```
// Magic number to identify valid filesystem
#define VCB_MAGIC 0x41534442  // "ASDB" in ASCII hex

// Volume Control Block structure
// Stored at block 0 on disk
struct VCB
    {
    unsigned int numBlocks;      4 Bytes // Total number of blocks in the volume
    unsigned int blockSize;      4 Bytes // Block size in bytes
    unsigned int blockIndexFSM;  4 Bytes // Index of the block containing the File S
    unsigned int blockIndexRootDirectory; 4 Bytes // Index of the block containing the
    unsigned int MagicNo;        4 bytes // Magic number identifier (signature)
    };
```

Yellow = Num Blocks; 4 Bytes.
Red = Block Size; 4 Bytes.
Green = Block index FSM; 4 Bytes.
Orange = BlockIndexRootDirectory; 4 Bytes.
Purple = MagicNo; 4 Bytes.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Volume Control Block Hex Dump



** Note: The bytes are in little-endian order so least significant bytes first.

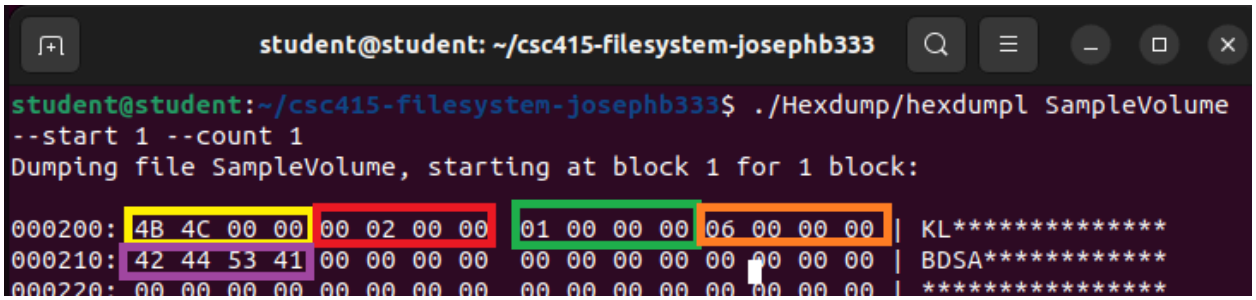
*** Note: At compile time intervals that don’t line up with 8 bytes become padded with 0’s.

Yellow = Num Blocks; Byte offset 0x0000 - 0x0003

0x00004C4B matches our expected number of blocks: 19531 in little endian

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Volume Control Block Hex Dump




```
student@student:~/csc415-filesystem-josephb333$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
Existing filesystem detected. Loading VCB...
VCB loaded: MagicNo=0x41534442, numBlocks=19531, blockSize=512, FSM=1, RootDir=6
|--------------------------------|
```

Yellow = Num Blocks; Byte offset 0x0000 - 0x0003

0x00004C4B matches our expected number of blocks: 19531 in little endian

Red = Block Size; Byte offset 0x0004 - 0x0007

0x00000200 matches our expected block size: 512 in little endian

Green = blockIndexFSM; Byte offset 0x0008 - 0x000B

0x00000001 reflects 1 in decimal, which matches our reflected LBA index for FSM at 1.

Orange = blockIndexRootDirectory; Byte offset 0x000C - 0x000F

0x00000006 is 6 as expected for our LBA index of Root Directory.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Volume Control Block Hex Dump

```
student@student: ~/csc415-filesystem-josephb333
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 1 --count 1
Dumping file SampleVolume, starting at block 1 for 1 block:

000200: 4B 4C 00 00 00 02 00 00  01 00 00 00 06 00 00 00 | KL**************
000210: 42 44 53 41 00 00 00 00  00 00 00 00 00 00 00 00 | BDSA************
000220: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

```
student@student:~/csc415-filesystem-josephb333$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
Existing filesystem detected. Loading VCB...
VCB loaded: MagicNo=0x41534442, numBlocks=19531, blockSize=512, FSM=1, RootDir=6
|--------------------------------|
```

Purple = MagicNo; Byte offset 0x0010 - 0x0013

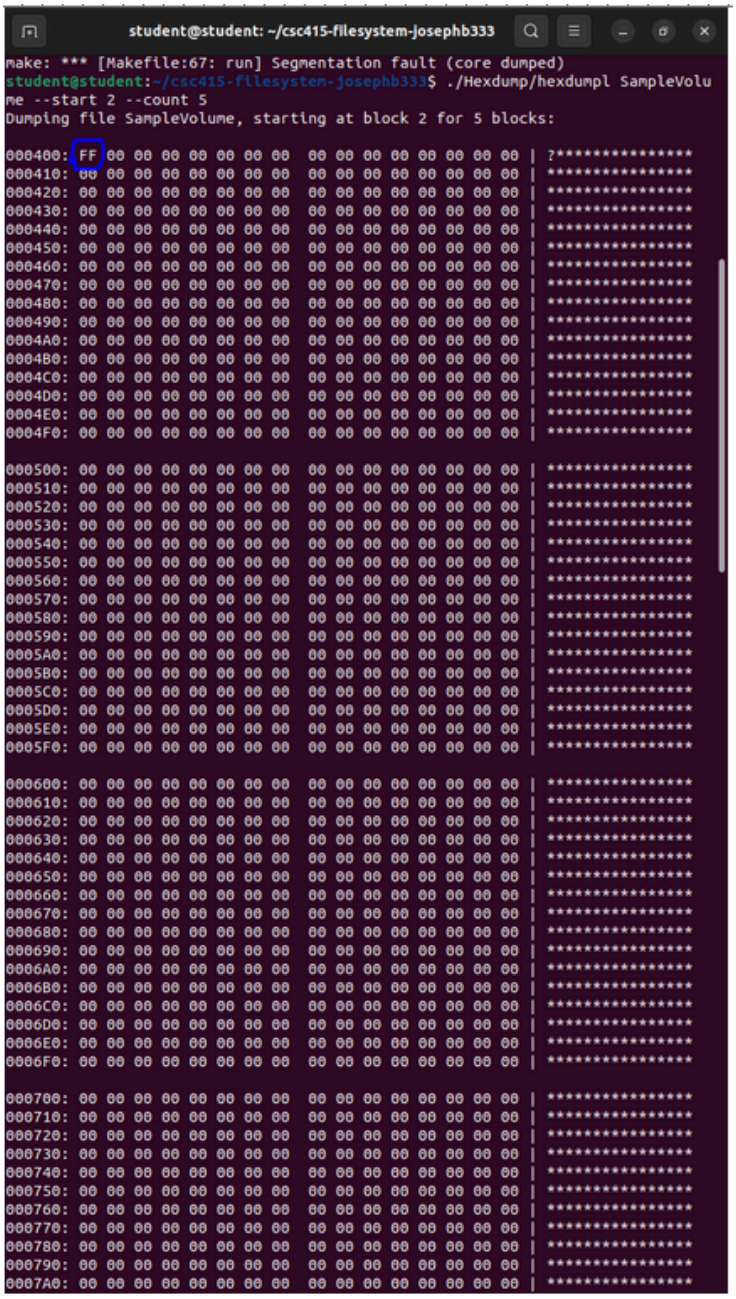
0x41534442 = Is the expected integer value we set as the magic number that corresponds to BDSA. Put into little endian order.




```
// Magic number to identify valid filesystem
#define VCB_MAGIC 0x41534442  // "ASDB" in ASCII hex
```

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

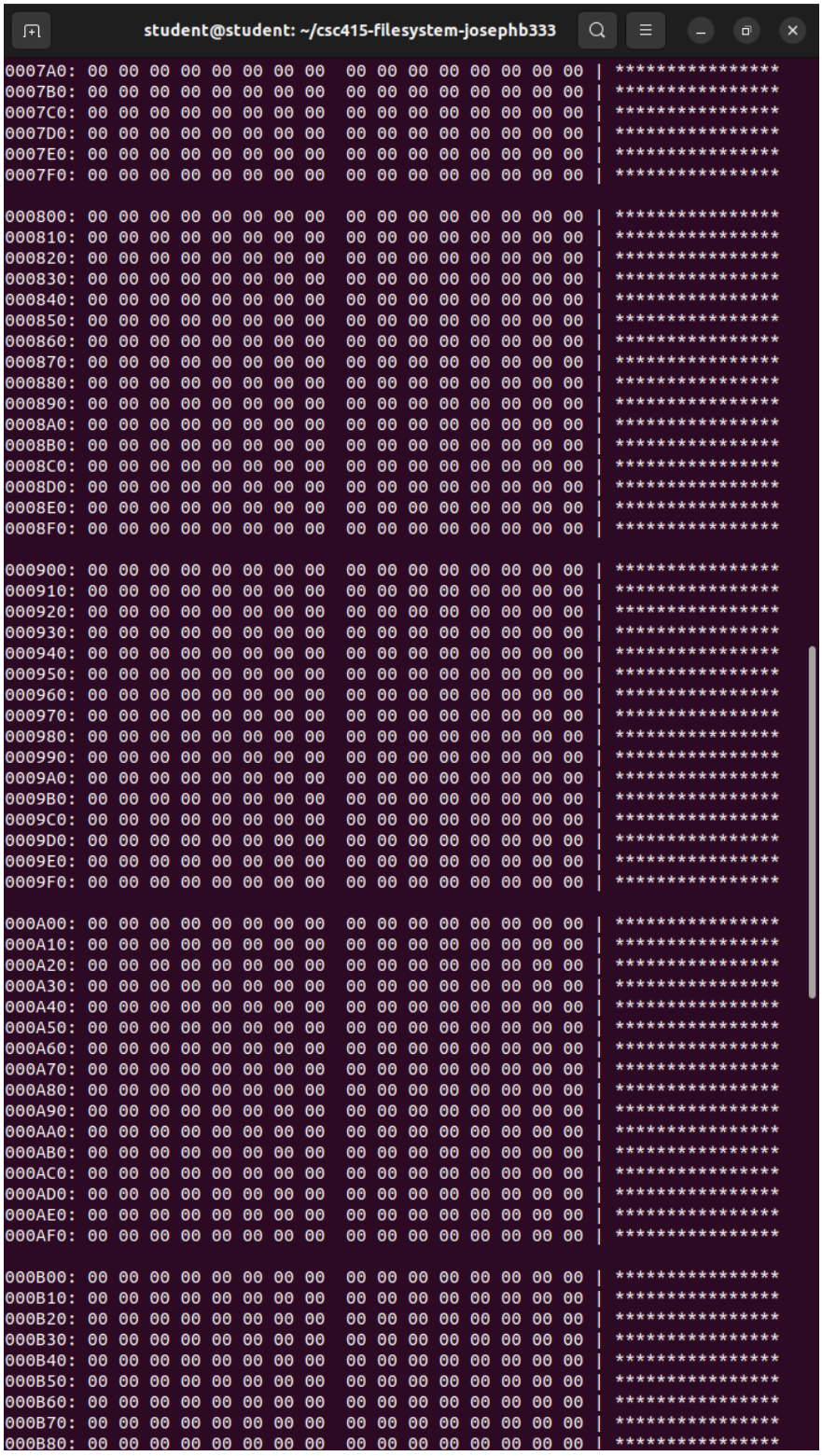
Free Space Map Hex Dump



^^ FF (11111111)b matches our expected free space map allocation, 1 block for the partition table, 1 block for the VCB, followed by 5 blocks for the FSM, and finally 1 block for the root directory for a total of 8 blocks.

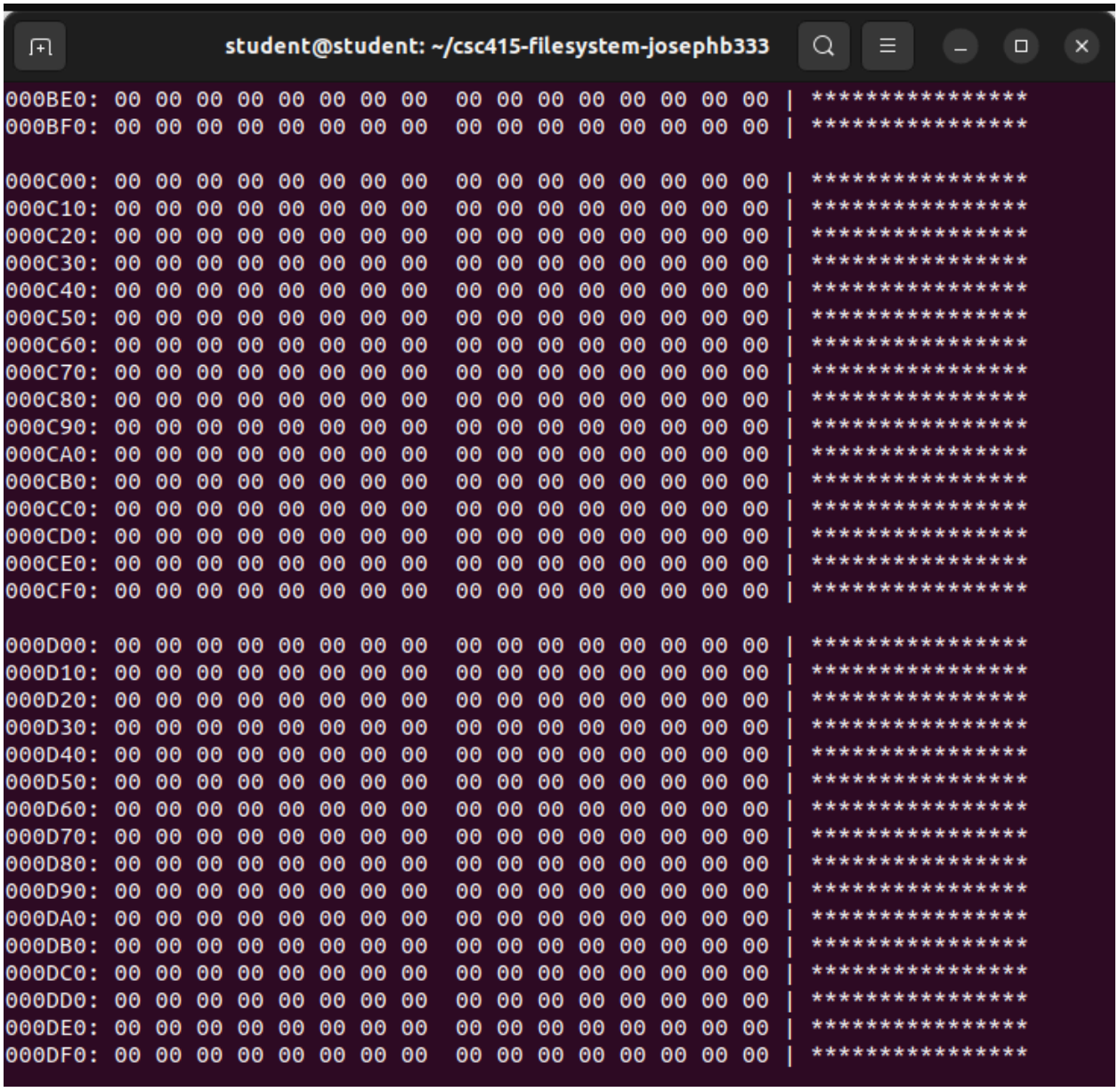
Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Free Space Map Hex Dump



Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Free Space Map Hex Dump



Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

In order to determine if our hex dump is correct we will first analyse the layout of our struct’s fields with corresponding byte lengths.

```
//Directory Entry Struct
typedef struct DE {
    // Starting head of the file's location (for discontiguous files)
    int blockNumber;        4 Bytes

    // File name
    char name[32];          32 Bytes

    // File size in bytes
    unsigned int numBytes;  4 Bytes

    // Flag to distinguish directory vs file
    int isDir;  // 1 for directory, 0 for ...   4 Bytes

    // Relevant Dates and Times (using time_t instead of char arrays)
    time_t createDate;
    time_t modifyDate;      8 Bytes each; 24 total
    time_t accessDate;

    // For discontiguous storage, we need to track the chain of blocks
    int loc;  // Starting location in block allocation structure
} DE;         4 Bytes
```

RED = Block Number; 4 Bytes;
ORANGE = name; 32 Bytes;
YELLOW = numBytes; 4 Bytes;
GREEN = isDir; 4 Bytes;
BLUE = createDate, modifyDate, accessDate; 8 Bytes each, 24 total.
PURPLE = Location; 4 Bytes;

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************

000F00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F30: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000F40: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000F50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F80: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000F90: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000FA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FD0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000FE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

** Note: The bytes are in little-endian order so least significant bytes first.

*** Note: At compile time intervals that don’t line up with 8 bytes become padded with 0’s.

RED = Block Number; Byte offset 0x0000 - 0x0003;
This is the block number of Directory. Which corresponds to 0x00000007 (in little-endian order) which is 7 which is where the physical block number we would expect the root dir. To be at, with Professor’s partition table at block 0, our VCB at block 1, the FSM occupying the next 5 blocks i.e. blocks 2 - 6.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

ORANGE = name; Byte offset 0x0004 - 0x0023;
The next 32 bytes are the name. Which is 2E 00 00… Which converted to text is “.” as expected.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

YELLOW = numBytes; Byte offset 0x0024 - 0x0027;
Next we have E0 01 00 00, Which when converted into little-endian order and hex is 0x000001E0 which when converted to decimal is 480 bytes. Now note here, we are creating two directories entries here, one with name “.” and the other with “..”

```
if (minEntries < 2) { minEntries = 2;}
int bytesNeeded = minEntries * (int)sizeof(DE);
//4d) How many blocks needed.
int blocksNeeded = (bytesNeeded + (blockSize - 1)) / blockSize;
int bytesToMalloc = blocksNeeded * blockSize;
// How many entries could fit in that space
int actualEntriesCount = bytesToMalloc / (int)sizeof(DE);
```

However, when we have sizeof(DE) which = 60 and blockSize = 512. Then we do integer division of 512 / 60 = 8.

```
    // Initialize all entries as empty
    for (int i = 2; i < actualEntriesCount; i++) {
        dir[i].name[0] = '\0';
        dir[i].isDir = 0;
        dir[i].numBytes = 0;
        dir[i].blockNumber = -1;
        dir[i].loc = -1;
    }
```

So we allocate 1 full block of directory entries to avoid external fragmentation to follow best practices. Which gives us 60 bytes per DE * 8 entries.
Which equals 480 which is exactly what we see in the hex dump

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

GREEN = isDir; Byte offset 0x0028 - 0x002B;

Here we are expecting 1 which is a sentinel value we picked to indicate if a DE is a directory or not here.

```
dir[0].isDir = 1;
```

And with little-endian order converting to hex we got 0x00000001 which is 1 in decimal as expected.

Note: There is padding included at compile-time between isDir and Dates because so far we have the blockNumber = 4 Bytes, name = 32 Bytes, numBytes = 4 Bytes, isDir = 4 Bytes. 4 + 32 + 4 + 4 = 44. To keep the next things aligned in memory the compiler does such in 8-byte values. So it pads the next 4 bytes to get to 48.

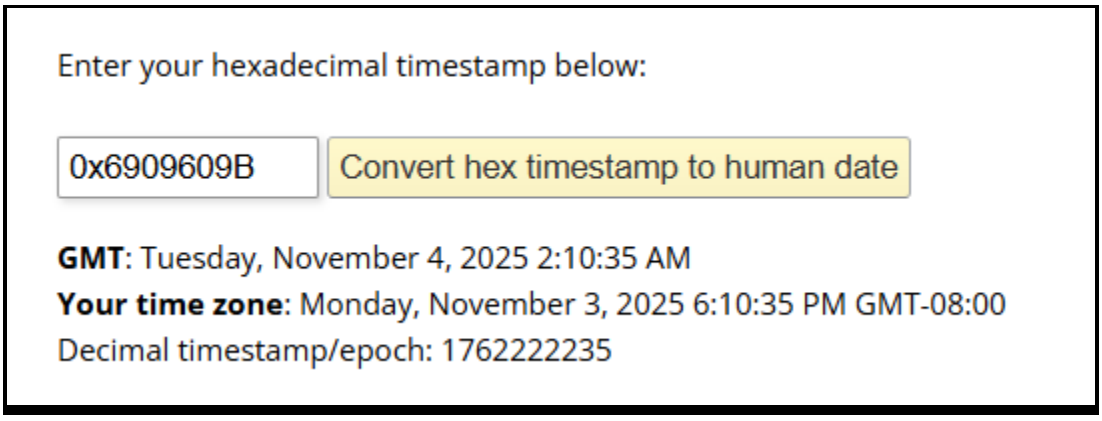
Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

BLUE = createDate, modifyDate, accessDate (all same for now).
Byte offset 0x0030 - 0x0047;
Here when we convert to little-endian order we get 0x6909609B.
Which when put into a hex-decimal to timestamp calculator. Is the expected value of when we created the directory.



**Note = We applied the same date to all fields for this assignment. That is why you see the same 9B 60 09 69 repeated three times for createDate, modifyDate, and accessDate.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

PURPLE = Location; Byte offset = 0x0048 - 0x004B;
Again we would expect the location to be in block 7 similar to our first field blockNumber. Which is what we see, as again we get 0x00000007 which is 7 when converted to decimal.
*Note: We are going to have two location variables one for the head, (blockNumber) and location of where the next block is for discontiguous tracking. Yet, at the current juncture we are not storing these directory entries discontiguous (since they both fit in one block) so they are both in the same block. Come next milestone we will implement a linked list data structure to facilitate the discontiguous allocation of directory entries.

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Root Directory Hex Dump

```
student@student:~/csc415-filesystem-josephb333$ ./Hexdump/hexdumpl SampleVolume
--start 7 --count 1
Dumping file SampleVolume, starting at block 7 for 1 block:

000E00: 07 00 00 00 2E 00 00 00  00 00 00 00 00 00 00 00 | ****.***********
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E30: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E40: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000E50: 07 00 00 00 2E 2E 00 00  00 00 00 00 00 00 00 00 | ****..**********
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 E0 01 00 00  01 00 00 00 00 00 00 00 | ****?***********
000E80: 9B 60 09 69 00 00 00 00  9B 60 09 69 00 00 00 00 | ?`*i****?`*i****
000E90: 9B 60 09 69 00 00 00 00  07 00 00 00 00 00 00 00 | ?`*i************
000EA0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  FF FF FF FF 00 00 00 00 | ********????****
000EF0: FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00 | ????************
```

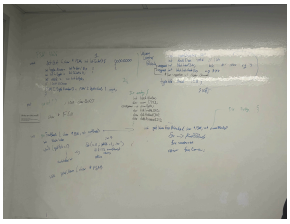
Note: The subsequent bytes in PINK are simply the next directory which we would expect to have all the same values except for a different name. Which is what we see, the whole block from 000E50 - 000E03 is the next directory. With the only difference being ↓↓↓↓

PINK = name (of “..” directory); Byte offset 0xe00 - 0xE03 following the same format.

* Note: 2E 2E is the name here corresponding to “..”

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Meeting Table:

Date	Attended	Purpose	Notes:
10/17	All	First thoughts of the project, how are we going to approach and what functions are we using	 ^Whiteboard notes
10/27	All	Coordinate who is working on what, and when testing should begin.	By Friday we should all be done writing code and have a group meeting during which we begin testing
10/31	All	Get everyone’s individuals’ parts functional to the point we could get a Hexdump.	Finished implementation and fixed all bugs to the point we got working hexdump. Still need to do: 1.Make sure formatted header is included with correct names, descriptions, student ids in ALL files. 2. Put in all of our issues and resolutions 3. Put in hexdump including analysis
11/2	All	Verify hex dump is correct/Resolve subsequent issues.	Finished Correcting Hex Dump. TO do's for tomorrow's meeting at 12pm. 1. Have all code done, including proper headers 2. Put correct hex dump 3. Clarify analysis of hex dumps 4. Put student ids at header of writeup 5. Put ALL of our issues and resolutions (try to include as many pertinent pictures as possible)

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

			Objective of meeting. Make sure writeup and code is good to go, push everything, and submit
11/3	All	Final checks	Took much more work than expected. More or longer meetings may be required in future. To Do’s: 1. Familiarize ourselves with necessary next steps so we can appropriately delegate future work for milestones. 2. Submit final writeup once everyone is finished.

Responsibilities

Joe	Ian	James	Enrique
Dir	FSM	Dir	VCB

Don’t Worry We Don't Byte: Ian Kligman, Joe Bowen, Cheng-Chieh Yao, Enrique Cabral-Mixco

Issues & Resolutions

We identified several critical issues in the original Directory Entry implementation that needed to be addressed to create a functional file system. The issues focused on five main areas: date storage, volume control structure, freespace management, entry type identification, and discontiguous block tracking.

The first issue was the date storage implementation. We decided to use time_t instead of character arrays to store dates. The original implementation used char arrays like createDate[] to store dates in DD/MM/YYYY format, which is format-dependent and difficult to manipulate. time_t is a standard type that stores dates as seconds since the Unix epoch, making date comparison and arithmetic much simpler. These date fields are now stored as time_t values. This approach is standard because it allows easy use of functions like time(), difftime(), and localtime() from the standard library, and eliminates format-dependency issues.

The second issue involved adding a VCB structure. We decided to implement a VCB to maintain file system metadata. The final rendition of the VCB structure contains essential information including number of blocks, block size, Logical Block Array LBA index of FSM, and magic no which is our signature.

```
// Volume Control Block structure
// Stored at block 0 on disk
struct VCB
    {
    unsigned int numBlocks;         // Total number of blocks in the volume
    unsigned int blockSize;         // Block size in bytes
    unsigned int blockIndexFSM;     // Index of the block containing the File System Metadata
    unsigned int blockIndexRootDirectory;  // Index of the block containing the Root Directory
    unsigned int MagicNo;           // Magic number identifier (signature)
    };
```

To distinguish between directories and files, we decided to add an isDir flag to the Directory Entry structure. The original DE structure had no way to identify whether an entry represented a file or a directory, which is essential for proper file system operations. The isDir field is an integer that is set to 1 for

directories and 0 for regular files. This flag allows the system to perform appropriate operations based on entry type - directories can be traversed and listed, while files can be read and written. Without this field, operations like recursive directory traversal would be impossible, and the system could not prevent invalid operations like trying to read a directory as if it were a file.

At first, we ran into the issue of not accounting for the professor’s partition table occupying physical block 0. The FSM needs to account for this offset, because it will assign blocks to those who call allocate. If it is off by one, the free space map will assign already used blocks to the caller! To correct this, we made sure to mark block 0 as used in the fsmInit. This way “our” block 0, where we understand the VCB to be assigned, will correctly be assigned to physical block 1 and so on.

The next issue was initially writing a needlessly complicated implementation of allocate(). This first version returned the starting block number of a group of n free blocks as requested by the caller. However,for our discontiguous allocation files, we only need to allocate to serve one free block at a time, regardless of whether or not these free blocks are next to each other. This first implementation would only be necessary if we were using contiguous files. To fix this we simplified the allocate() function to simply return one-at-a-time free blocks.

The first issue I ran into involved the two of us (James and I) working on the same part (Root Directory). We were having trouble coordinating so we both ended up making different changes to DE struct and methods which made it confusing to iterate through.

I resolved this by utilizing the steps that Professor had posted and going through his code step by step to make sure that it was doing what it was correctly and that it wasn’t missing any parts. By having a structured lens through which I could reference the work being done it made it way easier to iterate upon his work.

The next issue I had was about the non-functional entanglement between the three pieces (VBC, FSM, and Root Dir). During our third meeting our goal was to get all of our individual parts to work such that we could get a visible hex dump.

I resolved it by realizing there had been confusion surrounding what parts should be where. For instance, there had been declarations (with no implementation) of functions that were in the FSM (Free Space Map) that had leaked into DirectoryEntry.c. Part of the issue solution involved #including the correct files in their respective places. Once we sorted out what needed what and where we finally got it to compile.

The next issue we faced was a deluge of warnings/errors we were getting once we compiled.

I resolved the issue by coordinating with each part’s respective parts and utilizing line itemized error feedback from the terminal to find, identify, and remediate errors one by one. After fixing about 20. We were finally about to get a fully compiled program with no errors.

The next issue we faced was the fact that our hexdump was displaying info contrary to what we expected it to do.

We were able to resolve it by carefully reading through the hexdump module and realizing that the Professor had a partition table at physical block 0. Once we realized that we displayed blocks 1, 2, and 7 respectively to see if our data was working correctly.

The next issue we faced was the fact that although we were able to see the data we wanted with the hex dumps, we realized that they were in the incorrect blocks. E.g. Block 7 which we expected to have the root directory was empty and was at block 8.

We resolved it by realizing that something was going wrong with the changes we made to account for physical block 0 being reserved for the partition table. Our logic to account for it throughout was inconsistent and unrobust. We had to go through all previous iterations to code to make sure that we updated all the necessary writes/reads to make sure they were looking at correct blocks. Also we had to fix the fact that the FSM was

missing load functionality. The reason this was critical was because we had set up initFSM() such that it had two paths: 1. If the signature isn’t recognized, we format the volume. 2. That it is and we simply load VCB and FSM. We were loading the FSM() in initFSM() but in the case that the file system wasn’t recognized it was not. As such the allocate() function we used in the FSM wouldn’t be able to give the correct available block number. Furthermore, we had previous functional remnants of code that still did not account for +1 translation between physical and logical block addresses. Also we realized that LBAwrite() correctly accounted for physical address input. So all we had to do was change all lbawrites() back to their physical address, and include a more robust path for a found file system and then finally our blocks were showing up in their correct places.